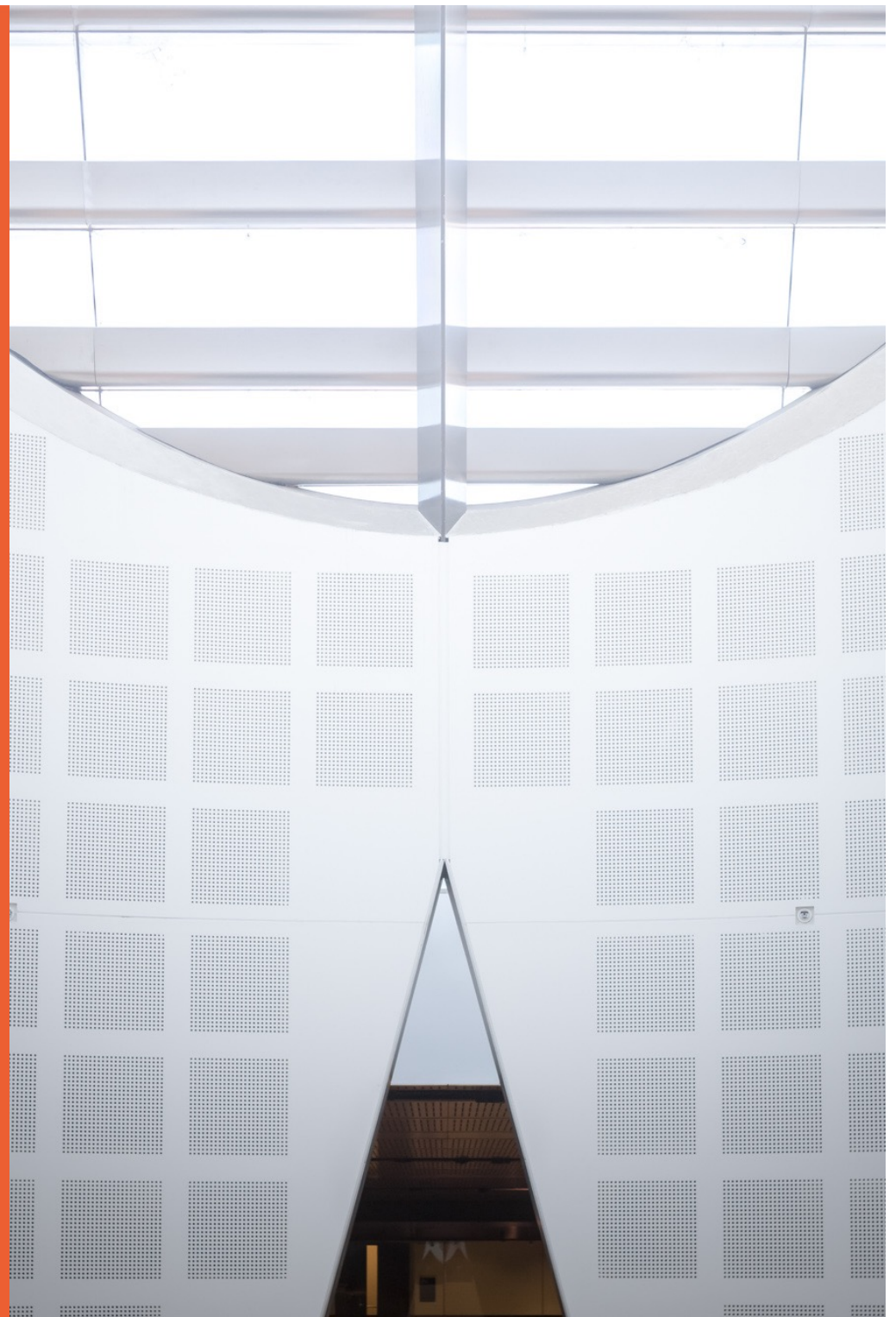


Web Servers and Application Architecture

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.

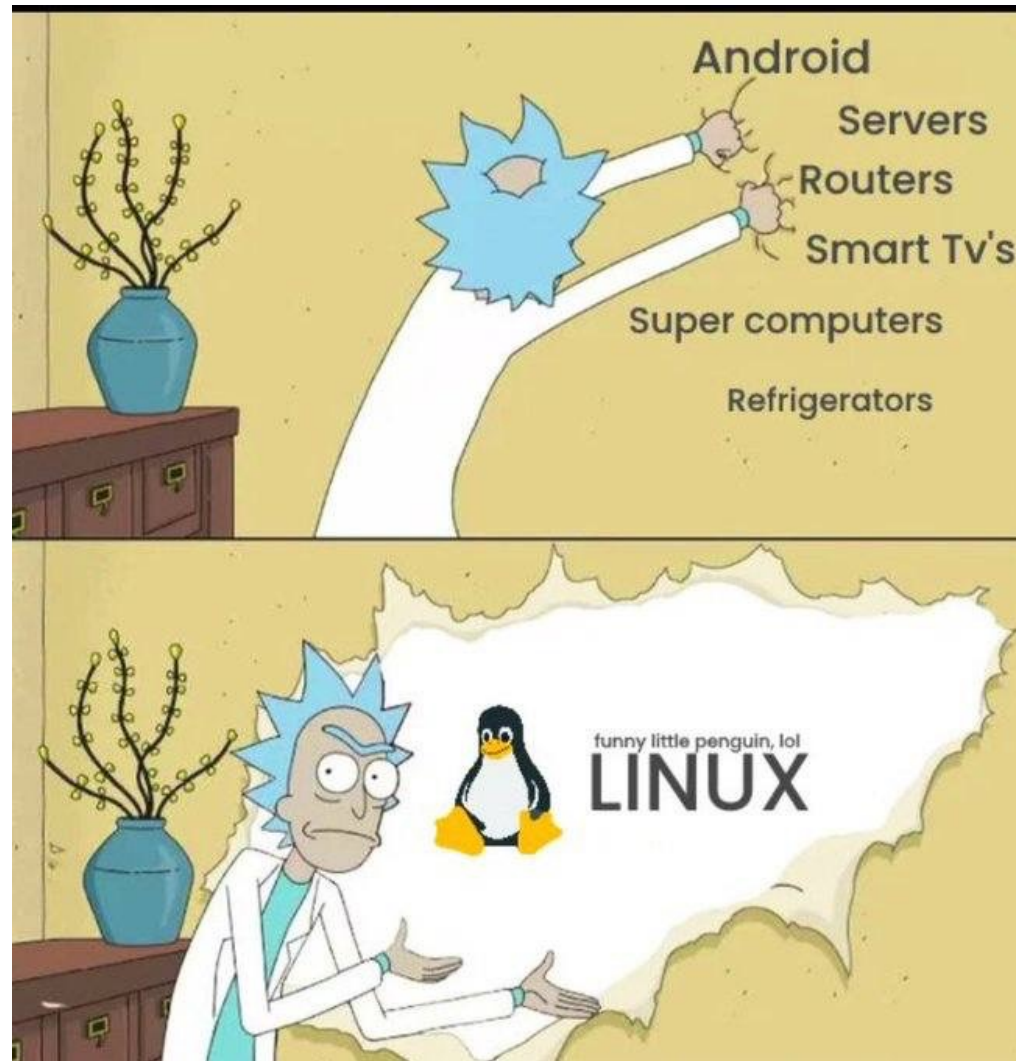


THE UNIVERSITY OF
SYDNEY

Overview

- ✓ **Web Architecture**
- ✓ **Web Servers**
- ✓ **Languages and Frameworks (Django)**
- ✓ **Data Storage**
- ✓ **Scaling Web Architecture**

Web Architecture



LAMP/LEMP

- LAMP / LEMP stacks are the most ubiquitous stacks used for developing and deploying web applications. Due to their modular nature, the components can easily be swapped out.

LAMP

- Linux
- Apache
- MySQL
- PHP

LEMP

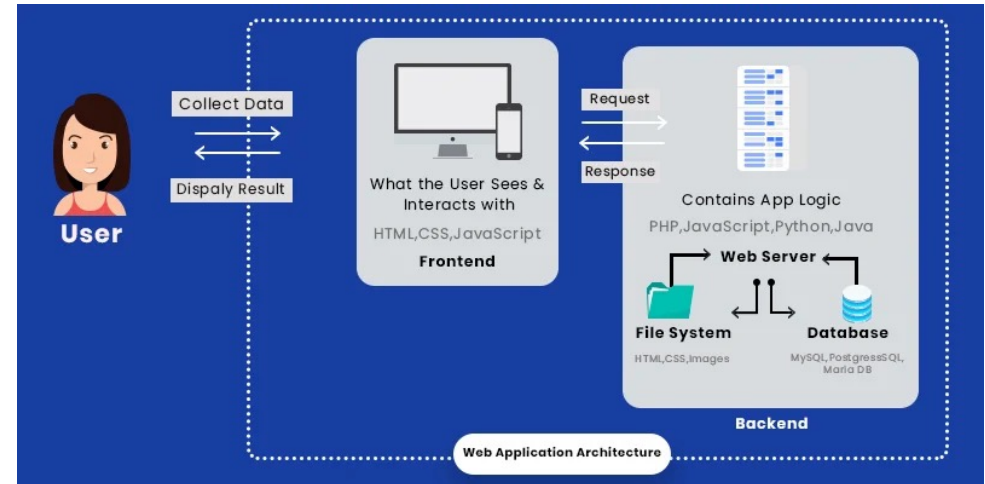
- Linux
- Nginx
- MySQL
- PHP

Linux Versions

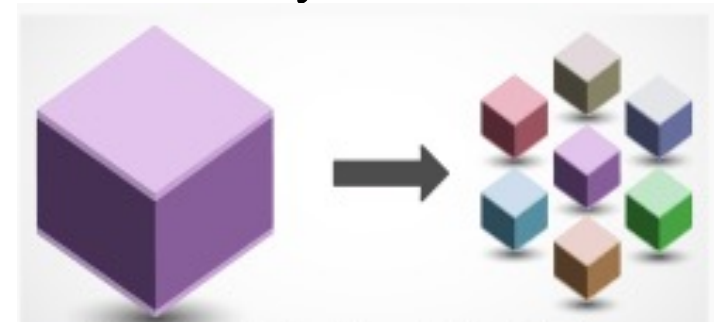
Make sure to use a well supported OS. e.g. if choosing [Ubuntu](#) use a Long Term Support (LTS) release. These are supported for 5 years.

Principled Design of the Modern Web Architecture*

- Modern web architecture



- Scalability of component interactions
- Generality of interfaces
- Independent deployment of components
- Intermediary components to reduce interaction latency
- Enforce security
- Encapsulate legacy systems.



- *: A paper published in 2002, <https://dl.acm.org/doi/pdf/10.1145/514183.514185>
- Images credits to internet

Single Server Architecture

Use Case

The entire environment resides on a single server. For a typical web application, that would include the web server, application server, and database server.

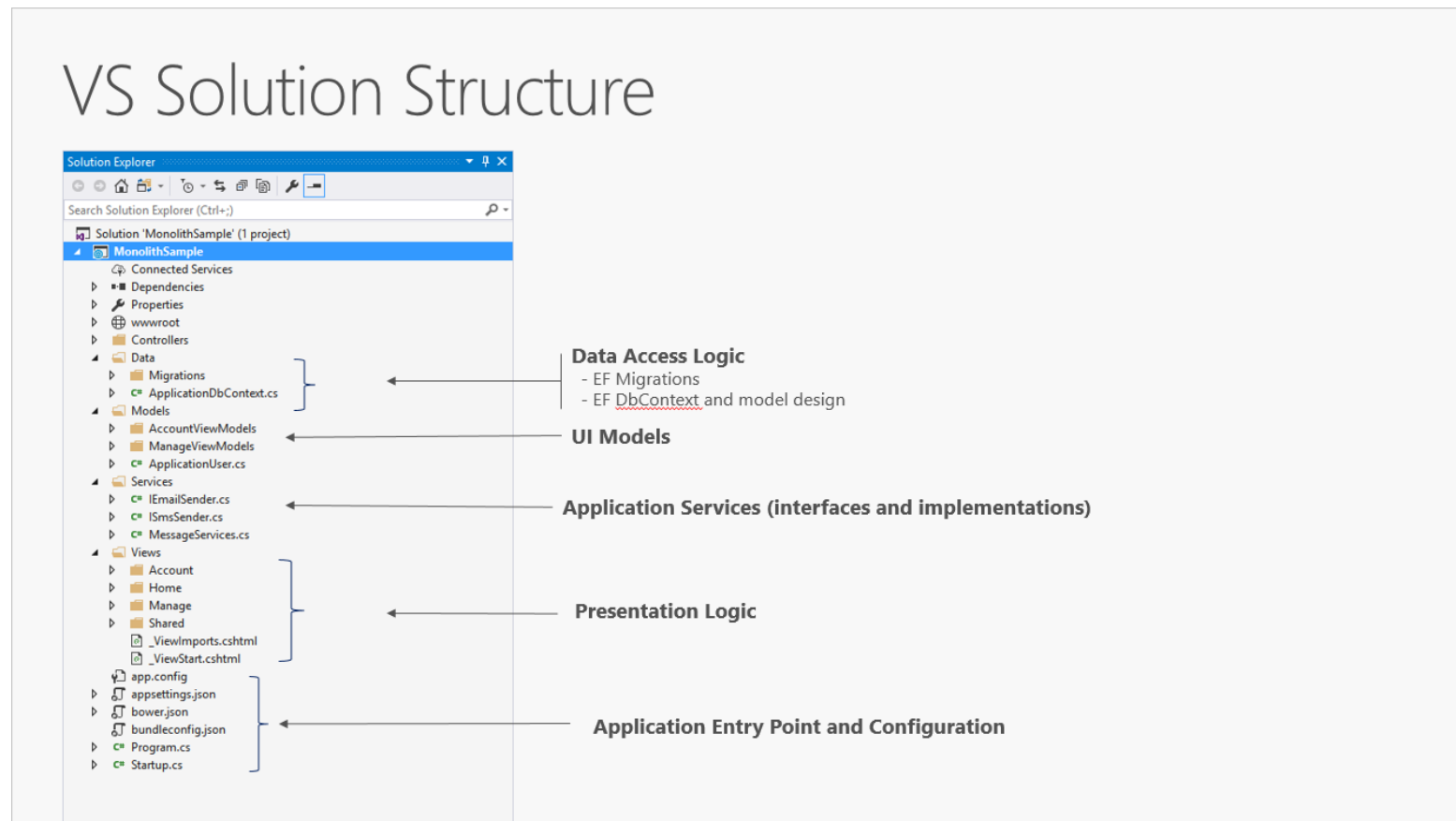
Good for setting up an application quickly, as it is the simplest setup possible, but it offers little in the way of scalability and component isolation.

Cons

Application and database contend for the same server resources (CPU, Memory, I/O, etc.) which, aside from possible poor performance, can make it difficult to determine the source (application or database) of poor performance. Not readily horizontally scalable

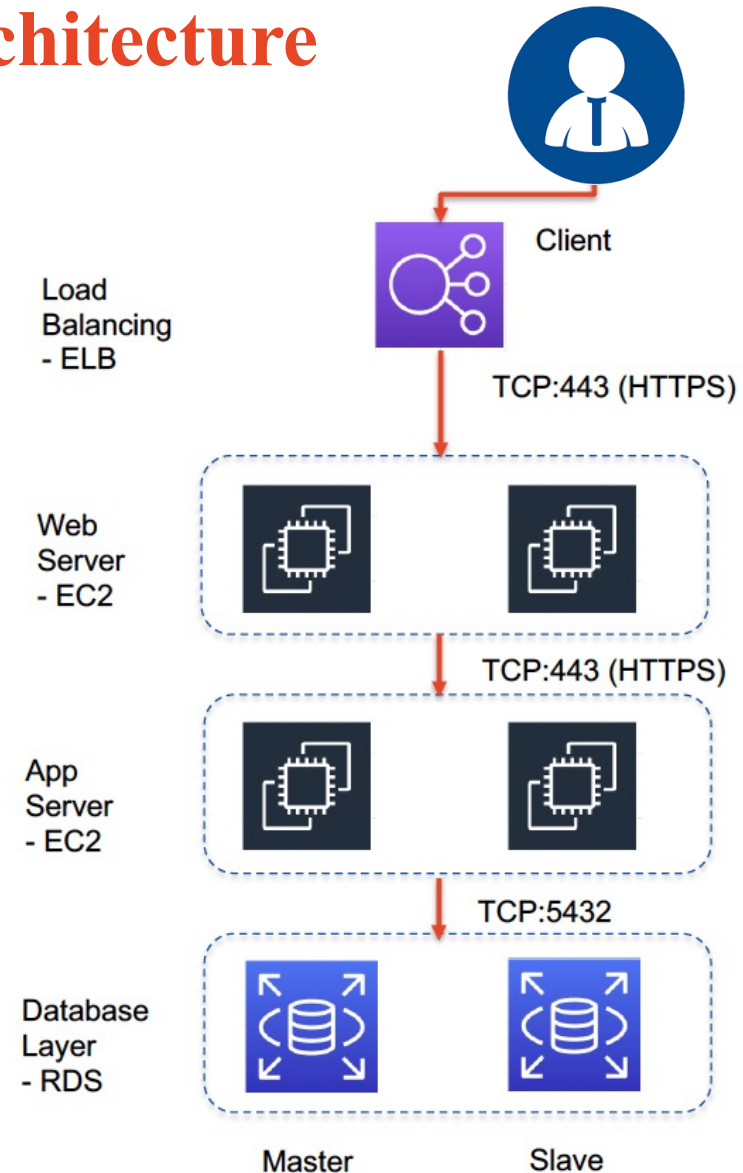
Single Server Architecture

- A new ASP.NET core project



Refreshing – Three layers architecture

- Web Server
 - Taking HTTP(S) traffic only
 - HTTP (port 80) or HTTPS (port 443) if encrypted
- Application Server
 - Possibly running on HTTP (80) or HTTPS (443) if encrypted
 - Or, any designated TCP ports



Terms– Application Server vs Web Server

- Web server exclusively handles HTTP requests, whereas an application server serves business logic to application programs through any number of protocols, possibly including HTTP.

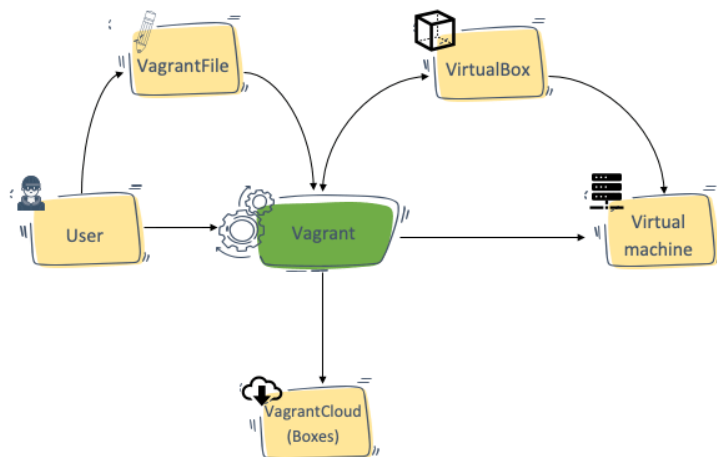
Example

Client <–> Web Server

Client <–> Web Server <–> Application Server

Example Website

- To setup a development webserver and develop a sample website.
- Always develop on a similar system to what you intend to deploy on. For us that means Linux.
- Vagrant is computer software that creates and configures virtual development environments.



Setup Ubuntu Server 14.04.5 LTS

```
$ vagrant init ubuntu/trusty64
```

```
# Modify Vagrantfile to forward port 80
```

```
$ vagrant up
```

```
$ vagrant ssh
```

MVP (Minimal Viable Product)

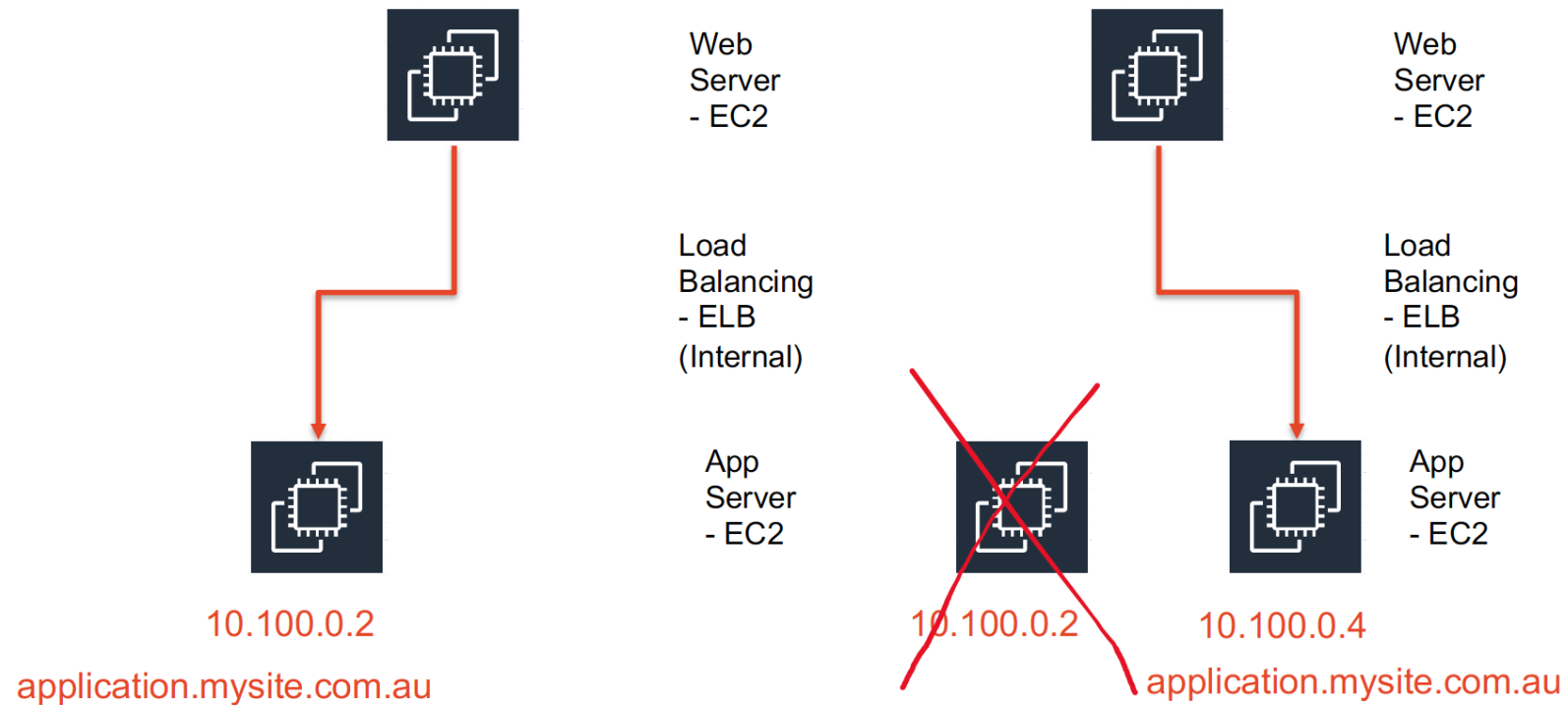
- Running everything on a single server

Pros	Cons
Simple for debugging, usually as the first iteration	Not production ready (real configuration would be very different when things on multiple layers and different compute nodes)
Good for demo or technical validation purposes	Architecturally things are not decoupled. Tough for troubleshooting
	Not ready for horizontal scaling

Case study - 1

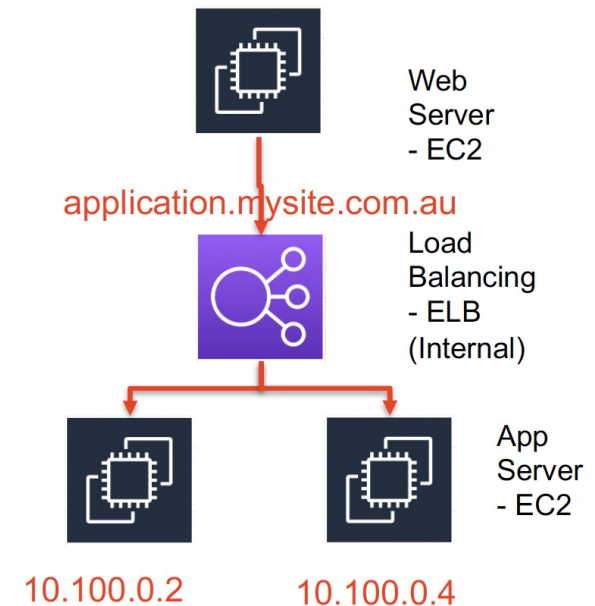
- In a single server MVP setup, your web server talks to the application server over this - localhost:8080
- If the application server is moved to another server, what would you do? What are the options?
 - IP address e.g. 10.100.0.2:8080
 - Issue: what if the server is dead, and a new server comes with a new IP address? Oh... this IP address is HARDCODED in one of the configuration files.
- DNS (A record)
 - A record : application.mysite.com.au is pointing to 10.100.0.2
 - Instead, now the web server configuration file says application.mysite.com.au
 - What if the application servers is dead? And the new one is assigned with a new IP address (by DHCP server) 10.100.0.4
 - Simple. Change the DNS record (can be achieved automatically with scripts) to the new IP
 - Application.mysite.com.au = 10.100.0.4
 - Web server does not need any configuration changes

Case study – 1 (continued)



Case study - 2

- Everything is fine. However, now the application server gets slammed.
- What would you do to improve its performance?
 - Option 1: Bigger instance with more CPUs (or a more powerful CPU) and with more memories.
 - Option 2: More instances
 - OK. Now problem comes. You have two application servers. 10.100.0.2 and 10.100.0.4
 - And you only have one representative DNS record for application server: application.mysite.com.au
 - What would you do?
 - Internal load balancer now kicks in.
 - Application.mysite.com.au points to the load balancer and it would then route traffic to one of the web servers.



Web Servers

IIS

- Internet Information Services (IIS) is Microsoft's web server.
- Released 1996.
- Long history of security vulnerabilities, and should be avoided if possible.
- Use it when running an ASP or ASP.NET application (still consider Apache) or to make use of Windows specific features such as AD-integration.

Apache

- Apache HTTP Server is an open source project now developed by the Apache Software Foundation.
- Very commonly used - released 1995.
- Dynamically loadable module system.
- Can scale but can be tricky to setup.
- Typically spawns threads to handle requests `mpm_prefork`. There are other options.

Nginx

- Released 2004 - designed from beginning to scale easily.
- Adoption has grown quickly.
- Spawns worker processes.
- Memory and CPU usage typically relatively consistent, even under load.
- Sample configuration file.

Example Website

- Once having an Ubuntu Server environment, a LEMP stack can be set up accordingly.

```
$ sudo apt-get update
$ sudo apt-get install nginx
$ sudo service nginx start
# Default web root is /var
```

```
# Define the "http" context
http {
    # Basic settings
    include mime.types;
    default_type application/octet-stream;
    sendfile on;

    # Configure the server
    server {
        # Listen on port 80
        listen 80;

        # Server name (domain or IP)
        server_name example.com www.example.com;

        # Location for serving static files
        location / {
            root /path/to/website; # Path to your static files
            index index.html;
        }

        # Reverse proxy configuration
        location /api/ {
            proxy_pass http://backend-server-ip:backend-server-port;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        # Error pages
        error_page 404 /404.html;
        error_page 500 502 503 504 /50x.html;

        location = /50x.html {
            root /path/to/error/pages;
        }
    }
}
```

Languages and Frameworks (Django)

HTML

HyperText Markup Language (HTML) is the standard markup language used to create web pages. The World Wide Web Consortium (W3C) develop HTML standards.

Example website

```
<HTML>
  <HEAD>
    <TITLE>Dilbert Quotes</TITLE>
  </HEAD>
  <BODY>
    <H1>Quote Of The Day</H1>
    <P>Engineers like to solve problems.
      If there are no problems handily
      available, they will create their
      own problems.</P>
  </BODY>
</HTML>
```

ASP and ASP.NET

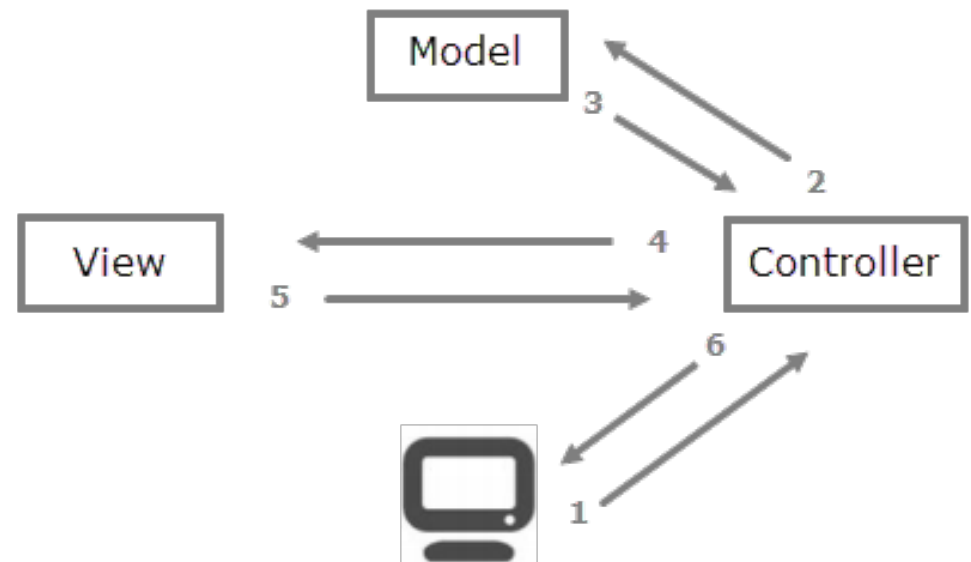
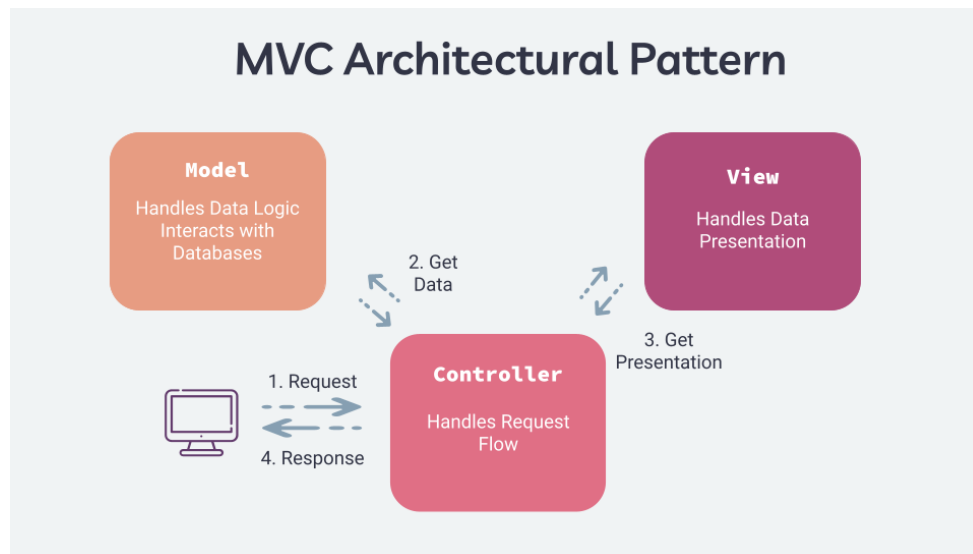
- Active Server Pages (ASP) was released in 1996 and is now known as Classic ASP or ASP Classic.
- Microsoft's first server-side script engine for dynamically generated web pages.
- ASP.NET (released 2002) supersedes ASP.
- ASP.NET uses a Common Language Runtime (CLR) to allow most .NET languages to be supported.

Python

- Use a framework.
- Language evolution planning using PEPs - you know what you get in the next few years:
 - Flask
 - Django
 - Typically come with built-in development server (lightweight web server)
- Setup with Apache & Nginx using WSGI (Web Server Gateway Interface) specification.

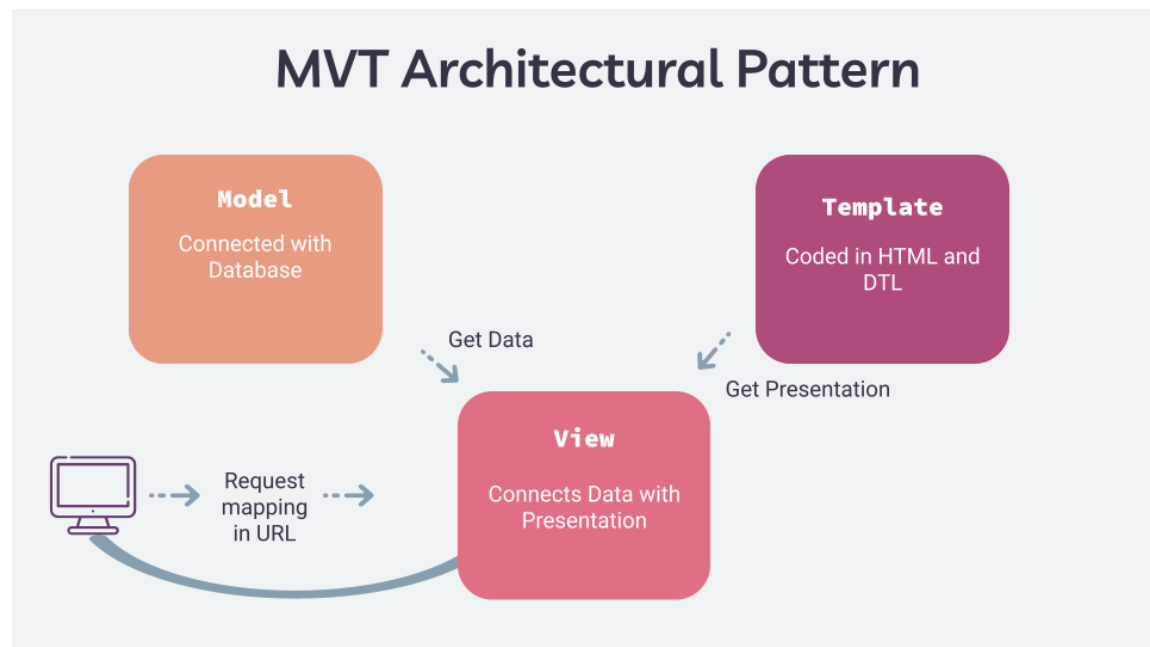
Django

- Models: Models represents how data is organized in the database.
- Views: A view is what you see when you visit a site.
- Controllers: Controller controls the flow of information.



Django

- Model, similar to MVC handles the data and logic part, maintains a connection with the database.
- View works differently than the View-layer of MVC. Here, the view accepts the requests and sends out the response.
- Template, this is the presentation layer. It is commonly coded in HTML or DTL (Django Template Language).



Images credits to the internet.

Python – Example Website

Django app - views.py

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Engineers like to solve  
problems. If there are no problems  
handily available, they will create their  
own problems.")
```

Django app - urls.py

```
from django.conf.urls import url

from . import views

urlpatterns = [
    url(r'^$', views.index, name='index'),
]
```

PHP

- Use a framework.
- Without proper planning it will become a mess (e.g. spaghetti code):
 - Codeigniter
 - CakePHP
- Can setup with either Apache or Nginx.

PHP– Example Website

Setup PHP with Nginx

```
$ sudo apt-get install php5-fpm
$ sudo vim /etc/nginx/sites-enabled/default
# Uncomment lines to enable passing PHP
  scripts to FastCGI server
$ sudo service nginx restart
# PHP setting file - /etc/php5/fpm/php.ini
# PHP-FPM setting file - /etc/php5/fpm/pool.
  d/www.conf
```

quote.php

```
<?php
echo("Engineers like to solve problems. If there
    are no problems handily available, they will
    create their own problems.");
```

Data Storage

Storage media

- Hardware form factor
 - Magnetic (spinning) drive vs. Solid State Drive (flash) HDD
 - Interconnected with Ethernet (iSCSI) or Fibre Channel - SAN storage
 - Network Attached Storage (NAS)
 - In-memory storage
- Object store
 - AWS S3
 - Google Cloud Storage
- Block store
 - Disks
- File store
 - NFS
 - Samba
 - CephFS
 - HDFS (Hadoop)

Disk

- Static files (HTML, Images, CSS).
- SSD vs Spinning.
- Leverage OS RAM caching.

Relational Databases

- Maintains and enforces relationships between data.
- Organizes data into one or more tables of columns and rows.
- Typically, each table represents an entity, such as a customer or product.
- Rows represent instances of that entity.
- Columns represent values attributed to that instance (such as address or price).
- Schema change often results in system downtime and planned maintenance

SQL – who needs a standard?

- There has been an ANSI / ISO SQL standard since 1986.
- Most SQL is not completely portable among RDBMS.
- Typically, only effects more advanced features e.g. Teradata doesn't support pivots.
- Transact-SQL (TSQL) is typically more feature rich.

MySQL/MariaDB

- Most popular and commonly used.
- Lots of third-party tools and libraries.
- Does not implement the full SQL standard.
 - e.g., 'foo' || 'bar' means 'foo' OR 'bar', whereas the standard defines || as string concatenation.
- Concurrency can be an issue.

Postgres

- ANSI/ISO SQL standards.
- Complete support for reliable transactions - Atomicity, Consistency, Isolation, Durability (ACID).
- Great spatial extension.
- Replication can be tricky.



Microsoft SQL Server (MSSQL)

- Good for MS stack.
- Integrates into BI/DW stack easily.
- SSIS, SSAS, SSRS.
- Generally cheaper than Oracle and Teradata.
 - Consider and plan license costs, even in the cloud
 - <https://www.zdnet.com/article/what-microsofts-upcoming-outsourcing-licensing-changes-could-mean-for-your-business/>
- DBAs.

New SQL or No SQL

- Non-relational databases store data more like loose pages, and does not store data in the same relational manner.
- New data structure
- More flexible schemas or no schemas
- Better scalability
- Eventually consistent rather than strictly consistent

Redis– K/V Based

- Written in: C - blazing fast.
- Protocol: Telnet-like, binary safe.
- Disk-backed in-memory database, master-slave replication, automatic failover.
- Simple values or data structures by keys.
- Supports INCR & co, bit and bitfield operations, sets, lists, hashes and sorted sets.
- Values can be set to expire (as in a cache),
- Pub/Sub lets you implement messaging.

Use Case

Use for rapidly changing data with a foreseeable database size (should fit mostly in memory).

For example: To store real-time stock prices. Real-time analytics. Leaderboards. Could be used as replacement for memcached.

Cassandra – Column Based

- Written in: Java.
- Main point: Store huge datasets in "almost" SQL.
- Protocol: CQL3 & Thrift.
- CQL3 is very similar to SQL, but with some limitations that come from the scalability (most notably: no JOINS, no aggregate functions).
- Querying by key, or key range (secondary indices are also available).
- Data can have expiration (set on INSERT).
- Map/reduce possible with Apache Hadoop.

Use Case

Use when you have massive amounts of data that can't fit on a single host, but still want a familiar interface to it.

For example: Web analytics, to count hits by hour, by browser, by IP.
Transaction logging. Data collection from huge sensor arrays.

MongoDB – Document Based

- Written in C++.
- Main point: JSON document store.
- Master/slave replication (auto failover with replica sets).
- Sharding built-in.
- Queries are JavaScript expressions.
- Geospatial queries and indexing.
- Multiple storage engines with different performance characteristics.
- Journaling.
- Text search integrated.
- Deployment would require some heavy-lifting if you want HA.
E.g. voting systems using ZooKeeper

Use Case

Use if you need dynamic queries or you want to define indexes, not map/reduce functions. Good performance on large databases.

For example: For most things that you would do with MySQL or PostgreSQL, but having predefined columns really holds you back.

ElasticSearch – Document Based

- Written in: Java.
- Main point: Advanced Search - stores JSON documents.
- Protocol: JSON over HTTP (Plugins: Thrift, memcached).
- Parent and children documents with versioning & time outs.
- Very versatile and sophisticated querying, scriptable.
- Write consistency: one, quorum or all. Asynchronous replication.
- Sorting by score, geo distance.
- Fuzzy searches (approximate date, etc).
- Use cases: dynamic web search with auto-suggest; ELK stack for log analysis
- ELK = ElasticSearch, LogStach, Kibana

Use Case

Use when you have objects with (flexible) fields, and you need advanced search functionality.

For example: A directory that handles skill differences, geographic location, online/offline, etc.

Cloud Storage

- Cloud storage is a model of data storage in which data is stored in logical pools, the physical storage spans multiple servers (and often locations), and the physical environment is typically owned and managed by a third party.
- Many offerings on the market: Azure Storage, Google Cloud Storage, Amazon S3.

Amazon S3

- Amazon Simple Storage Service (S3) is a cost effective, simple to use service that you can be used to store and retrieve any amount of data, at any time, from anywhere on the web.

AWS API

```
# Creating a bucket
```

```
$ aws s3 mb s3://bucket-name
```

```
# Copy local file to bucket
```

```
$ aws s3 sync some-local-file s3://bucket-name/  
path
```

Best Practices of Web Architecture in Production

Decoupling

Give different components dedicated resources

- Web Servers
- App Servers
- Database

Good to have a well-understood scope of work

Reduced blast radius

Easier troubleshooting and incident response

Smaller but more components. Moving towards micro services architecture

Scalability

- Statelessness: this is the foundation of all scalability. Stateful applications won't scale.
 - Components of the same layer should not share anything in between them
- Refer to these principles: 12 Factor App <https://12factor.net/>
- Scale out vs. scale up
 - Scale-out: horizontal scaling; The model supercomputing has been using; **load balancing is IMPORTANT**
 - Scale-up: bigger, bigger and bigger. Often not realistic due to the physical laws and economics
- Scalability does not apply to only compute and memory, but also IO
 - IOPS (IO per second)
 - Bandwidth
 - Latency

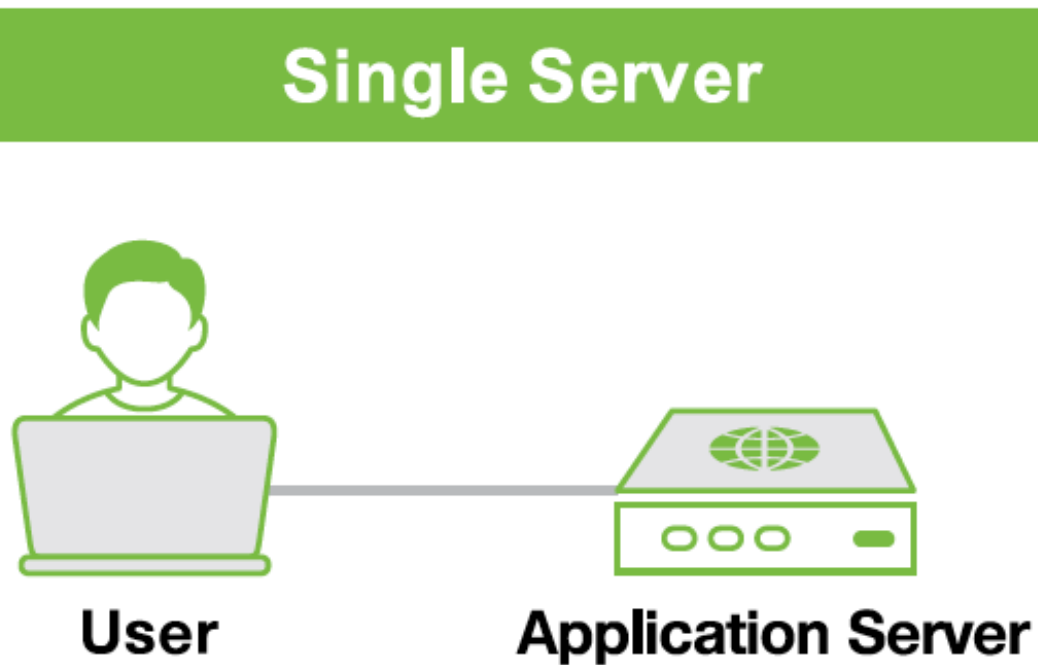
Redundancy

- Remove all SPOFs (single point of failure)
 - **Master/Slave; Replications**
 - Take a guess on AWS S3's durability
- Durability vs. Availability vs Reliability vs Resilience
 - Durability: is the data stored securely in the event of failures?
 - Availability: refers to system uptime. Percentage of time able to serve requests.
 - Reliability: similar to availability. System can be available but not functioning correctly
 - Resiliency: ability for the system to recover in the event of failures. Self-healing.
- Some software defects are in relation to performance (cannot be replicated when the load is low)
- Think about Avalanche or cascading effects.
- Extended reading: [Chaos Monkey](#)

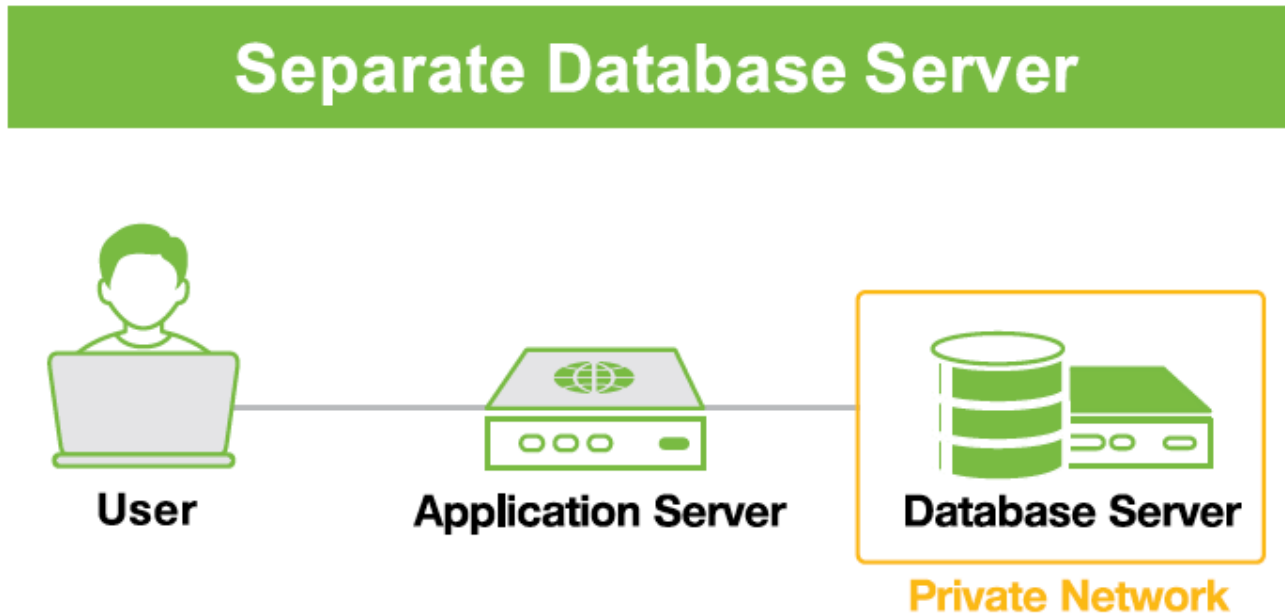


Scaling Web Architecture

Recap: Single Server Architecture



Separate DB Server



- Separate the database from the application server to eliminate resource contention. Increase security by removing the database from the DMZ, or public internet.

Separate DB Server

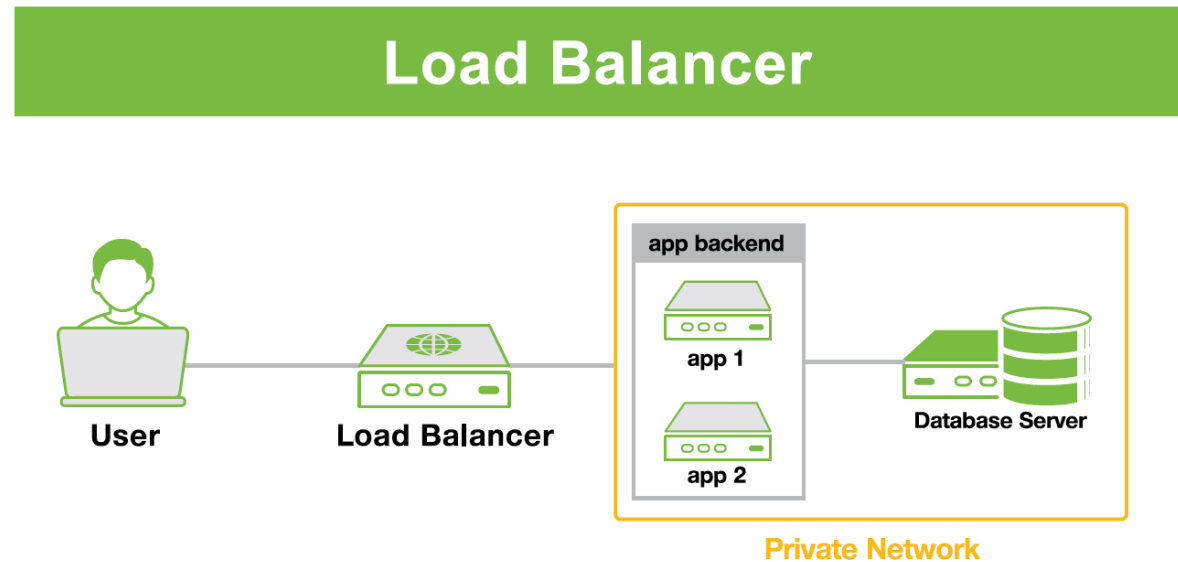
Pros

- ⦿ Application and database tiers do not contend for the same server resources (CPU, Memory, I/O, etc.)
- ⦿ You may vertically scale each tier separately
- ⦿ Can increase security by removing your database from the DMZ

Cons

- ⦿ More complex setup than single server
- ⦿ Performance issues can arise if the network connection between servers is poor

Load Balancer



- Load balancers (LB) can be added to a server environment to improve performance and reliability by distributing the workload across multiple servers. If one of the servers that is load balanced fails, the other servers will handle the incoming traffic until the failed server becomes healthy again.
- Typical implementations use HAProxy, Nginx, and Varnish.

Load Balancer

Pros

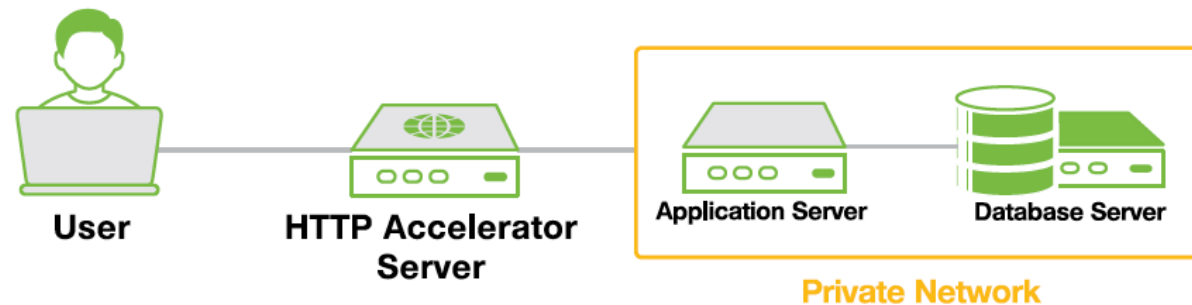
- ⦿ Useful in an environment that requires scaling by adding more servers (horizontal scaling).
- ⦿ Can protect against DDOS attacks by limiting client connections to a sensible amount and frequency

Cons

- ⦿ The LB can become a performance bottleneck if it does not have enough resources, or if it is configured poorly
- ⦿ Can introduce complexities that require additional consideration, e.g. where to perform SSL termination
- ⦿ The LB is a single point of failure - if it goes down, your whole service can go down

HTTP Accelerator

HTTP Accelerator



- A HTTP accelerator, or caching HTTP reverse proxy, can be used to reduce the time it takes to serve content to a user through a variety of techniques. Typically the response from a web or application server is cached for a period of time in memory.
- Examples of software capable of HTTP acceleration: Varnish, Squid, Nginx.

HTTP Accelerator

Pros

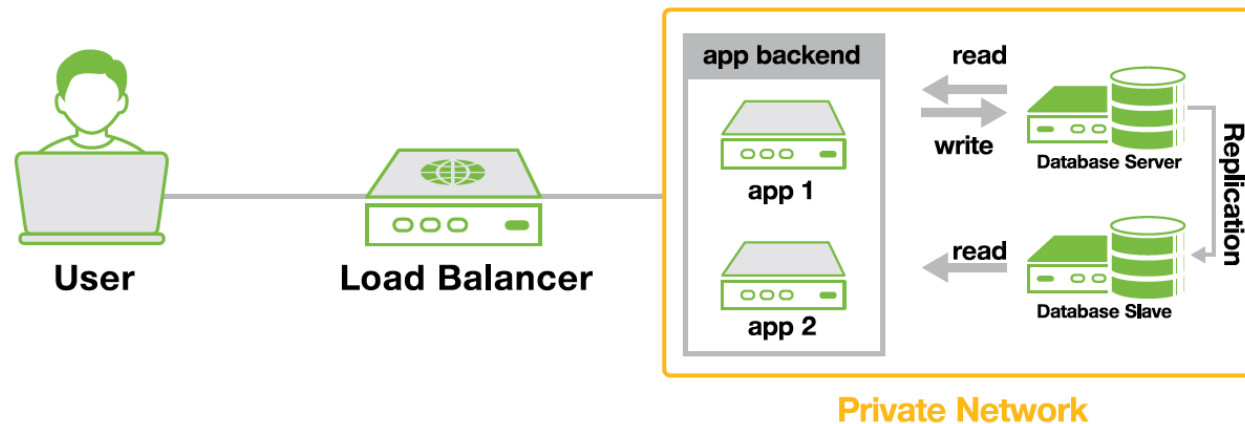
- ⦿ Increase site performance by reducing CPU load on web server, through caching and compression, thereby increasing user capacity
- ⦿ Can be used as a reverse proxy load balancer
- ⦿ Depending on implementation, can protect against DDOS attacks

Cons

- ⦿ Requires lots of tuning
- ⦿ If the cache-hit rate is low, it can reduce performance
- ⦿ Can be a single point of failure

Master-Slave Database Replication

Master-Slave Database Replication



One way to improve performance of a database system that performs many reads compared to writes, such as a CMS, is to use master-slave database replication. Master-slave replication requires a master and one or more slave nodes. In this setup, all updates are sent to the master node and reads can be distributed across all nodes.

Master-Slave Database Replication

Pros

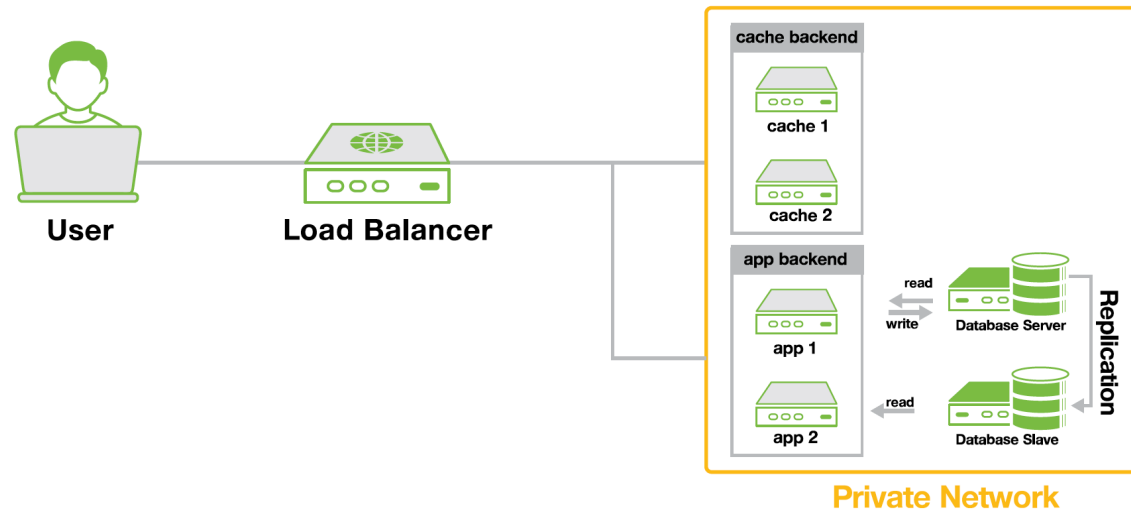
- ⦿ Improves database read performance by spreading reads across slaves
- ⦿ Can improve write performance by using master exclusively for updates

Cons

- ⦿ The application must determine which database nodes it should send update and read requests to
- ⦿ Updates to slaves are asynchronous - slave drift
- ⦿ If the master fails, no updates can be performed until the issue is corrected
- ⦿ Does not typically have built-in failover in case of failure of master node

Combinations

Load Balancer + Cache + Replication Example



It is possible to load balance the caching servers, in addition to the application servers, and use database replication in a single environment. The purpose of combining these techniques is to reap the benefits of each without introducing too many issues or complexity.

**You reached the end of the lecture.
Thank you and see you next week.**